WIDS : Market Mood & Moves : Sentiment-driven stock prediction

Week 1

Project Overview

This project, Market Mood & Moves: Sentiment-Driven Stock Prediction, captures my learning journey over four weeks, exploring how stock prices, financial news, and investor sentiment can be combined to understand and predict market behavior.

I started by learning how to collect real financial data and news using APIs, and how NLP can turn raw text into meaningful sentiment signals. I explored sentiment analysis tools like VADER and FinBERT, and realized why domain-specific models matter for financial text.

Later, I focused on time-series modeling, understanding why stock prices are sequential data and how models like RNNs and LSTMs capture temporal dependencies better than standard neural networks.

In the final stage, I built an end-to-end pipeline combining stock prices and news sentiment to generate Buy, Sell, or Hold signals. Overall, this project reflects my step-by-step understanding of how sentiment, market data, and deep learning come together in a practical trading system, along with the challenges involved.

Natural Language Processing Fundamentals

NLP helps convert unstructured text, such as news articles and headlines, into a form that machines can understand. In this project, NLP was used to extract meaningful signals from financial news and study how language reflects market sentiment.

Text Preprocessing

Tokenization : splitting text into words

Stopword removal : removing common but uninformative words

Lemmatization : reducing words to their base form

Text preprocessing cleans raw text and removes noise so that sentiment models focus on meaningful words. This step is crucial before applying any NLP or sentiment analysis technique.

API-Based Data Collection

Tools used:

NewsAPI for financial news headlines

Finance for historical stock price data

APIs were used to collect real-world, live data instead of relying on static datasets. This helped simulate realistic market analysis workflows and understand how financial data pipelines work in practice.

0.1 Sentiment Analysis

VADER Sentiment Analyzer

Characteristics are Lexicon-based, Rule-driven, Suitable for short text

VADER was used as a baseline sentiment analysis tool. It works well for short texts like headlines and provides sentiment scores that are easy to interpret.

FinBERT for Financial Text

FinBERT is a Domain-specific language model with context-aware embeddings, trained on financial text.

FinBERT is better than traditional sentiment models by understanding financial terminology and context. It assigns sentiment more accurately to finance-related text compared to general-purpose NLP models.

0.2 Quantitative Finance Fundamentals

Slippage Modeling

Slippage refers to the difference between the expected price and the actual execution price. I learned that fast markets, large orders, or low liquidity can cause trades to execute at worse prices, reducing profits.

Transaction Costs Analysis (TCA)

Even small fees like brokerage, taxes, and exchange charges can significantly impact performance. This concept taught me why frequent trading strategies may look profitable on paper but fail in real life.

Liquidity & Execution Probability

Liquidity measures how easily a stock can be bought or sold without affecting its price. I learned the trade-off between market orders (guaranteed execution, uncertain price) and limit orders (fixed price, uncertain execution).

Performance Metrics

I learned that evaluating a strategy requires measuring risk along with returns:

Maximum Drawdown (MDD): Shows the worst loss from peak to bottom — helps understand downside risk.

CAGR: Represents the average annual growth of an investment over time.

Win/Loss Ratio & Profit Factor: Measure consistency and overall profitability of trades.

Alpha & Beta: Explain how a strategy performs relative to the market and how risky it is.

Sharpe Ratio: Measures return per unit of risk — higher means better risk-adjusted performance.

Sortino Ratio: Similar to Sharpe but focuses only on harmful (downside) risk.

Calmar Ratio: Compares returns with maximum drawdown, useful during volatile markets.

Week 2

Week 2 Theoretical mastery

Q) Explain why Static Embeddings fail on the word "Bank" and the importance of BERT architecture.

Ans) Static embeddings like Word2Vec maps a word like "Bank" to one fixed vector but some words including bank can have multiple meanings and there the issue arises, Word2Vec takes a weighted average of those two vectors corresponding to different meanings.

BERT uses dynamic embedding, here embedding is a function. It goes through the entire sentence and understands the context. Hence, in financial text, BERT interprets bank as financial entity while Word2Vec will confuse.

Q) Draw the 3 components of the BERT input embedding.

ans) Final Input = Token + Segment + Position embeddings

Token Embeddings WordPiece breaks unknown words into subwords, solving out-of-vocabulary problem

Segment embeddings these tell which sentence a word belongs to

Position embeddings they tell about BERT word order, which word is positioned where , usually transformers don't understand sequence by themselves

Q) Explain the 80-10-10 Masking Rule

ans) 80-10-10 masking rule stands for

80 % mask : replacing word with MASK

10% random word : teaches the model to trust context

10% original word : prevents model from thinking that every input is wrong

Week 2 FinBERT specifies

Q) Understand the 3-stage training pipeline.

ans) Stage 1 : FinBERT is BERT, which is specially trained for finance

So here we do general language pre training like it learns english grammar and structure

Stage 2 : Domain Adaption, Trained on finn=ance documents, learns finance specific meanings

Stage 3 : Fine tuned for Sentiment analysis, learns to label text as positive, negative and neutral

Q) Define "Domain Adaptation" in the context of NLP.

ans) Domain Adaption is simply taking a general language model and retraining it on text from specific domain like finance

Example : FinBERT is BERT trained in finance

BERT understands bank broadly but FinBERT understands mainly as financial institution

Q) Explain why we use TRC2-Financial for pre-training and Financial PhraseBank for fine-tuning.

ans) We use TRC2-Financial for pre-training because it contains large volumes of real financial news and reports. This helps the model learn financial language, terminology, and context at a broad level.

We use Financial PhraseBank for fine-tuning because it has short financial sentences with clear sentiment labels (positive, negative, neutral). This helps the model learn how to correctly classify sentiment, which is the final task we care about.

Week 3

Sequential Nature of Stock Prices

One of the main takeaways was that stock prices cannot be treated like random numbers. Today's price is influenced by what happened earlier, which is why past data matters so much when trying to make predictions

Thinking in Probabilities

The reading helped me understand that predicting stock prices is really about estimating probabilities based on past behavior. Instead of exact values, the focus is on how likely a certain movement is, given previous trends

Autoregressive Models and Markov Assumption

I learned how autoregressive models use recent past values to predict the future. The Markov assumption makes things simpler by limiting how much history is considered, but it also means some long-term information can be missed

Why Normal Neural Networks Don't Work

Standard neural networks look at each input separately and don't remember anything. This made it clear why they fail for stock data, where the order and history of prices are very important

Recurrent Neural Networks (RNNs)

RNNs improve on this by carrying information forward using hidden states. This allows them to learn patterns across time, which makes them more suitable for financial time-series data

Vanishing Gradient Problem

While RNNs are better, they still struggle with learning long-term patterns. During training, important information from earlier time steps slowly fades, which limits their effectiveness

Long Short-Term Memory (LSTM) Networks

LSTMs solve this issue by deciding what information to keep or forget using gates. This helped me understand why LSTMs are commonly used for stock prediction and other sequence-based problems

Combining Price and Sentiment Data

The reading showed that market movements are not driven by prices alone. Adding sentiment from news helps the model capture investor behavior and emotional reactions in the marke

Finally, I learned that model outputs need to be converted into clear decisions like Buy, Sell, or Hold. This step is important because raw predictions alone are not directly useful in real trading.

Theoretical Mastery

Difference between Markov Models and RNNs

Markov models assume the current state depends only on a fixed number of past steps

They use a fixed-size sliding window of previous values

RNNs maintain a hidden state that summarizes the entire past sequence

RNNs do not require explicitly choosing a window size

Markov models are simpler but lose long-term context

RNNs learn what historical information is important automatically

Internal Structure of an LSTM Cell

LSTM separates long-term memory (cell state) from short-term output (hidden state)

Input Gate: Controls how much new information enters the cell

Forget Gate: Decides what old information should be removed

Helps adapt to changing market conditions

Output Gate: controls what part of the memory is exposed as output

Candidate Memory: proposes new information to be added to the cell state

Vanishing Gradient Problem in RNNs

RNNs are trained using Backpropagation Through Time

Gradients are multiplied repeatedly across time steps

If weights are small, gradients shrink exponentially

This causes early time steps to have almost no influence

As a result, standard RNNs forget long-term dependencies

LSTMs solve this by allowing gradients to flow through the cell state

Implementation

Sliding Window Dataset (PyTorch Dataset Class)

Implemented a custom PyTorch Dataset class

Uses a sliding window of fixed length (e.g., 60 days)

Each input contains a sequence of historical features

Target is the next-day return

Enables supervised learning on time-series data

Ensures temporal ordering is preserved

Sentiment LSTM Model with Dropout

Implemented an LSTM-based neural network in PyTorch Used multiple LSTM layers for better representation learning

Applied dropout to reduce overfitting

Used a fully connected layer to generate final predictions

Model follows a many-to-one architecture

Output represents predicted next-day returns

Model Training

Trained the model on more than one year of historical stock data

Included both technical features and sentiment features

Used Mean Squared Error (MSE) as the loss function

Used Adam optimizer for stable convergence

Applied gradient clipping to avoid exploding gradients

Observed learning behavior through training loss trends

Project Integration

Loading FinBERT and LSTM Models

Loaded the FinBERT model trained in Week 2

Used FinBERT to generate daily sentiment scores

Loaded the trained LSTM model for price prediction

Ensured both models operate in evaluation mode

Combined outputs from both models in a single pipeline

Mock Buy/Sell

Selected 5 technology stocks: AAPL, MSFT, GOOGL, NVDA, TSLA

Fetched latest stock price data using yFinance

Collected recent financial news using NewsAPI

Computed sentiment scores using FinBERT

Generated LSTM-based return predictions

Produced Buy / Sell / Hold signals based on combined logic

Trading Strategy Logic

Sentiment-Weighted Momentum Strategy

LSTM provides a technical prediction (expected return)

FinBERT provides a sentiment-based confidence signal

BUY signal: LSTM predicts upward movement, sentiment score is positive

SELL signal: LSTM predicts downward movement, sentiment score is negative

HOLD signal: Signals are weak or conflicting

Strategy reduces false trades and improves interpretability

Execution Workflow

Scraped financial news from the last 24 hours

Ran FinBERT to compute sentiment score

Fetched latest OHLCV market data

Updated technical features

Fed sequence window + sentiment into LSTM

Generated next-day prediction

Converted prediction into trading signal

Simulated order execution with basic risk control

Week 4

The following is a little brief of whatever I have done in week 4, putting it all together

News Collection and Sentiment Processing

I started by fetching real-time financial news using the NewsAPI. These headlines were then passed through a sentiment analysis step inspired by FinBERT, which turned qualitative news into numerical sentiment scores. This helped me get a feel for how market mood can actually be quantified.

Stock Price Data Collection

For stock prices, I used the yfinance API to get real historical OHLCV data. Using actual market data instead of simulated values made the pipeline feel much closer to a real trading system.

Feature Engineering Integration

Next, I combined price-based features, like returns and volume signals, with the sentiment scores. This step really helped me understand how both market behavior and human emotions can be represented together in a single feature set.

Sliding Window Sequence Creation

To feed the LSTM model, I organized the features using a sliding window approach. This made sure each prediction considered a fixed number of past days, reinforcing the idea that temporal context is key in financial prediction.

LSTM-Based Prediction

These sequences were then fed into an LSTM model to predict the next-day return. This step brought together everything I had learned about time-series modeling and deep learning in a practical way.

Decision Logic: Buy / Sell / Hold

Finally, instead of just using the model output directly, I added a decision layer. The predicted return was combined with the sentiment strength to generate Buy, Sell, or Hold signals, making the system more practical and somewhat risk-aware.

*** I had also written some concepts in jupyter notebooks markdown cells and as comments

Resources used

- API based data collection – https://newsapi.org/
- NLP fundamentals – https://www.nltk.org/book/
- For revising python – https://www.w3schools.com/python/
- For Pandas –
  - https://pandas.pydata.org/docs/user$_guide$/10$min.html$
  - https://www.geeksforgeeks.org/pandas-tutorial/
- For numpy – https://www.w3schools.com/python/numpy/
- Reading guides by mentors –
  - http://localhost:8888/lab/tree/week$_1$/$week_{1r}eading.pdf$
  - http://localhost:8888/lab/tree/week$_2$/$week_{2r}eading.pdf$
  - http://localhost:8888/lab/tree/week$_3$/$week_{3r}eading.pdf$
- For integrating into end to end pipeline, I used internet little here and there.